

Hash Tables: Complete Guide with Collision Resolution Techniques

Introduction to Hash Tables

A **hash table** (also called hash map or dictionary) is one of the most powerful and widely-used data structures in computer science, providing efficient key-value pair storage and retrieval^{[173][176]}. Unlike arrays that use indices, hash tables use **keys** to directly access values, enabling average-case $O(1)$ time complexity for insertion, deletion, and search operations^{[164][170][^182]}.

Fundamental Concepts

Hash Table Definition:

A hash table is a data structure that implements an associative array abstract data type, mapping keys to values using a hash function^{[164][173]}. It consists of:

- **Buckets/Slots:** Array positions where elements are stored
- **Keys:** Unique identifiers for data
- **Values:** Actual data associated with keys
- **Hash Function:** Converts keys into array indices

Core Principle:

The hash function transforms a key (of any type) into an integer index within the array bounds^{[170][164]}. This enables direct access to the corresponding bucket, bypassing sequential search.

Hash Functions

A **hash function** is a mathematical function that maps data of arbitrary size to fixed-size values (hash codes or indices)^{[164][173]}. An ideal hash function should:

- Be deterministic (same key $\rightarrow$ same hash)
- Distribute keys uniformly across buckets
- Be computationally efficient $O(1)$
- Minimize collisions

Common Hash Function Techniques:

1. Division Method (Modulo):

```
hash(key) = key % table_size
```

Simple and fast, but requires choosing `table_size` carefully (prime numbers preferred)^{[164][172]}.

2. Multiplication Method:

```
hash(key) = floor(m * (key * A mod 1))
```

Where $0 < A < 1$ (Knuth suggests $A \approx 0.618034$)

3. String Hash Functions:

- **djb2 Algorithm:** $\text{hash} = \text{hash} * 33 + \text{char}$ [166][178]
- **Polynomial Rolling:** $\text{hash} = (\text{hash} * \text{prime} + \text{char}) \% \text{table_size}$ [178]

4. Universal Hashing:

Randomly selected from a family of hash functions to guarantee good average-case performance [173].

Collision Phenomenon

A **collision** occurs when two different keys hash to the same index [171][174]. This is inevitable due to the **Pigeonhole Principle**: if the key space is larger than the table size, collisions must occur.

Collision Rate Factors:

- Hash function quality
- Table size (load factor)
- Key distribution
- Collision resolution strategy

Load Factor (α):

$$\alpha = n / m$$

Where n = number of elements, m = table size [177][180]

- $\alpha < 0.5$: Excellent performance
- $\alpha \approx 0.7-0.8$: Good balance for open addressing
- $\alpha > 0.9$: Performance degrades significantly
- $\alpha > 1.0$: Impossible for open addressing

Collision Resolution Techniques

There are two primary categories of collision resolution [171][174][168]:

Category 1: Separate Chaining (Open Hashing)

Concept: Each bucket contains a linked list (or other data structure) of all elements that hash to that index [174][178].

Characteristics:

- Keys stored both inside and outside hash table

- Number of keys can exceed table size
- Simple insertion and deletion
- Extra memory for pointers required^[168]

Advantages:

- Simple implementation
- No clustering problems
- Graceful degradation as load factor increases
- Can handle load factor > 1
- Easy deletion (just remove from linked list)

Disadvantages:

- Additional memory for pointers
- Poor cache performance (scattered memory access)^[168]
- Some buckets remain empty (space wastage)

Time Complexity:

- Average case: $O(1 + \alpha)$ where α is load factor
- Worst case: $O(n)$ if all keys hash to same bucket
- Search/Insert/Delete: $O(1)$ expected, $O(n)$ worst case

Category 2: Open Addressing (Closed Hashing)

Concept: All elements stored directly in the hash table itself. When collision occurs, probe for alternative locations^{[165][177][180]}.

General Probing Formula:

$$h(k, i) = (h'(k) + f(i)) \% m$$

Where:

- $h'(k)$ is the primary hash function
- $f(i)$ is the probing function ($i = 0, 1, 2, \dots$)
- m is table size

Characteristics:

- All keys stored inside hash table
- Number of keys $\leq$ table size
- More complex deletion (use tombstone markers)^[180]
- Better cache performance^[168]

- All buckets can potentially be used

Advantages:

- No extra memory for pointers
- Better cache locality
- No wasted buckets
- Memory-efficient

Disadvantages:

- Requires larger table (load factor < 1)
- Deletion is complex (requires lazy deletion)^[180]
- Clustering can occur
- Performance degrades rapidly as table fills

1. Linear Probing

Formula: $h(k, i) = (h(k) + i) \% m$

Algorithm:

1. Compute initial index: $index = hash(key) \% table_size$
2. If slot is occupied, try next slot: $index = (index + 1) \% table_size$
3. Repeat until empty slot found or table full^[174]^[180]

Example:

Table size = 7, $hash(x) = x \% 7$

Insert: 50, 70, 76, 85, 93

```
50: 50 % 7 = 1 → table[1] = 50
70: 70 % 7 = 0 → table[0] = 70
76: 76 % 7 = 6 → table[6] = 76
85: 85 % 7 = 1 (collision!) → try table[2] = 85
93: 93 % 7 = 2 (collision!) → try table[3] = 93
```

Primary Clustering Problem:

Consecutive occupied slots form clusters, increasing probe length^[177]^[180]. New insertions near clusters extend them, degrading performance.

Time Complexity:

- Best case: $O(1)$
- Average case: $O(1/(1-\alpha))$
- Worst case: $O(n)$

2. Quadratic Probing

Formula: $h(k, i) = (h(k) + c_1 \cdot i + c_2 \cdot i^2) \% m$

Common: $h(k, i) = (h(k) + i^2) \% m$

Algorithm:

1. Compute initial index: $\text{index} = \text{hash}(\text{key}) \% \text{table_size}$
2. If occupied, try: $(\text{index} + 1^2) \% \text{table_size}$, then $(\text{index} + 2^2) \% \text{table_size}$, etc.^[174]^[180]

Example:

Table size = 7, $\text{hash}(x) = x \% 7$

Insert: 22, 30, 50

```
22: 22 % 7 = 1 → table[1] = 22
30: 30 % 7 = 2 → table[2] = 30
50: 50 % 7 = 1 (collision!)
    → try (1 + 12) % 7 = 2 (collision!)
    → try (1 + 22) % 7 = 5 → table[5] = 50
```

Secondary Clustering:

Keys with same initial hash follow same probe sequence (milder than primary clustering)^[177].

Requirement:

Table size should be prime and load factor < 0.5 to guarantee finding empty slot^[180].

Time Complexity:

- Average case: $O(1/(1-\alpha))$
- Less clustering than linear probing

3. Double Hashing

Formula: $h(k, i) = (h_1(k) + i * h_2(k)) \% m$

Where:

- $h_1(k)$ = primary hash function
- $h_2(k)$ = secondary hash function (must not evaluate to 0)^[174]^[180]

Common Secondary Hash:

```
 $h_2(k) = \text{prime} - (k \% \text{prime})$ 
```

Where $\text{prime} < \text{table_size}$

Example:

Table size = 7

$$h_1(k) = k \% 7$$

$$h_2(k) = 1 + (k \% 5)$$

Insert: 27, 43, 692, 72

```

27:  $h_1(27) = 6 \rightarrow \text{table}[^6] = 27$ 
43:  $h_1(43) = 1 \rightarrow \text{table}[^1] = 43$ 
692:  $h_1(692) = 6$  (collision!)
       $h_2(692) = 1 + (692 \% 5) = 3$ 
       $\rightarrow \text{try } (6 + 1 \cdot 3) \% 7 = 2 \rightarrow \text{table}[^2] = 692$ 
72:  $h_1(72) = 2$  (collision!)
       $h_2(72) = 1 + (72 \% 5) = 3$ 
       $\rightarrow \text{try } (2 + 1 \cdot 3) \% 7 = 5 \rightarrow \text{table}[^5] = 72$ 

```

Advantages:

- Minimal clustering (different probe sequences)
- Better distribution than linear/quadratic
- Uniform probing

Disadvantages:

- More computation (two hash functions)
- Requires careful selection of $h_2[^{177}]$

Time Complexity:

- Average case: $O(1/(1-\alpha))$
- Best performance among probing methods

Comparison: Separate Chaining vs Open Addressing

Aspect	Separate Chaining	Open Addressing
Storage	Inside + outside table	Only inside table
Max Keys	Can exceed table size	$\leq$ table size
Memory	Extra for pointers	No extra memory
Cache	Poor (scattered)	Good (contiguous)
Deletion	Easy	Complex (tombstones)
Load Factor	Can be > 1	Must be < 1
Clustering	None	Possible
Implementation	Simple	Moderate
Best For	Unknown size, frequent deletes	Fixed size, cache-sensitive

Real-World Applications

Hash tables are fundamental to numerous applications^{[173][182]}:

Programming Languages:

- Python dictionaries
- Java HashMap
- C++ unordered_map
- JavaScript objects

Database Systems:

- Indexing for fast lookups
- Join operations
- Query optimization

Caching:

- Web caching (URL → content)
- DNS resolution
- Memoization in dynamic programming

Compilers:

- Symbol tables
- Variable name resolution
- Keyword recognition

Operating Systems:

- File system indexing
- Memory management
- Process scheduling

Network Applications:

- Router tables
- Load balancing
- Duplicate detection

Cryptography:

- Password storage (hash + salt)
- Digital signatures
- Blockchain

Performance Analysis

Time Complexity

Operation	Average Case	Worst Case
Insert	$O(1)$	$O(n)$
Search	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$

Worst case occurs when all keys collide (poor hash function or adversarial input).

Space Complexity

- Separate Chaining: $O(n + m)$ where n = keys, m = table size
- Open Addressing: $O(m)$ where $m \geq n$

Dynamic Resizing

When load factor exceeds threshold (typically 0.7-0.75):

1. Create new table (typically $2\times$ size)
2. Rehash all existing keys into new table
3. Replace old table with new one

Amortized Cost: $O(1)$ per operation despite $O(n)$ resize cost^{[173][176]}.

Implementation 1: Separate Chaining in C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

#define TABLE_SIZE 10
#define LOAD_FACTOR_THRESHOLD 0.75

// Node for linked list (chaining)
typedef struct Node {
    char* key;
    int value;
    struct Node* next;
} Node;

// Hash table with separate chaining
typedef struct {
    Node** buckets;
    int size;
    int count;
} HashTable;
```



```

// Hash function (djb2 algorithm for strings)
unsigned long hash_function(const char* str, int table_size) {
    unsigned long hash = 5381;
    int c;

    while ((c = *str++)) {
        hash = ((hash << 5) + hash) + c; // hash * 33 + c
    }

    return hash % table_size;
}

// Create new node
Node* create_node(const char* key, int value) {
    Node* node = (Node*)malloc(sizeof(Node));
    if (!node) return NULL;

    node->key = strdup(key);
    node->value = value;
    node->next = NULL;

    return node;
}

// Initialize hash table
HashTable* create_hash_table(int size) {
    HashTable* table = (HashTable*)malloc(sizeof(HashTable));
    if (!table) return NULL;

    table->size = size;
    table->count = 0;
    table->buckets = (Node**)calloc(size, sizeof(Node*));

    if (!table->buckets) {
        free(table);
        return NULL;
    }

    return table;
}

// Calculate load factor
float load_factor(HashTable* table) {
    return (float)table->count / table->size;
}

// Insert key-value pair
bool insert(HashTable* table, const char* key, int value) {
    unsigned long index = hash_function(key, table->size);
    Node* current = table->buckets[index];

    // Check if key already exists (update value)
    while (current) {
        if (strcmp(current->key, key) == 0) {
            current->value = value;

```

```

        printf("Updated: %s = %d\n", key, value);
        return true;
    }
    current = current->next;
}

// Insert new node at head of chain
Node* new_node = create_node(key, value);
if (!new_node) return false;

new_node->next = table->buckets[index];
table->buckets[index] = new_node;
table->count++;

printf("Inserted: %s = %d at bucket[%lu]\n", key, value, index);
return true;
}

// Search for key
bool search(HashTable* table, const char* key, int* value) {
    unsigned long index = hash_function(key, table->size);
    Node* current = table->buckets[index];

    while (current) {
        if (strcmp(current->key, key) == 0) {
            *value = current->value;
            return true;
        }
        current = current->next;
    }

    return false;
}

// Delete key
bool delete(HashTable* table, const char* key) {
    unsigned long index = hash_function(key, table->size);
    Node* current = table->buckets[index];
    Node* prev = NULL;

    while (current) {
        if (strcmp(current->key, key) == 0) {
            if (prev) {
                prev->next = current->next;
            } else {
                table->buckets[index] = current->next;
            }

            free(current->key);
            free(current);
            table->count--;
            printf("Deleted: %s\n", key);
            return true;
        }
        prev = current;
        current = current->next;
    }
}

```

```

        current = current->next;
    }

    return false;
}

// Display hash table
void display(HashTable* table) {
    printf("\n=== Hash Table (Separate Chaining) ===\n");
    printf("Size: %d, Count: %d, Load Factor: %.2f\n\n",
        table->size, table->count, load_factor(table));

    for (int i = 0; i < table->size; i++) {
        printf("Bucket[%d]: ", i);
        Node* current = table->buckets[i];

        if (!current) {
            printf("Empty\n");
        } else {
            while (current) {
                printf("(%s: %d) -> ", current->key, current->value);
                current = current->next;
            }
            printf("NULL\n");
        }
    }
    printf("\n");
}

// Free hash table
void free_hash_table(HashTable* table) {
    for (int i = 0; i < table->size; i++) {
        Node* current = table->buckets[i];
        while (current) {
            Node* temp = current;
            current = current->next;
            free(temp->key);
            free(temp);
        }
    }
    free(table->buckets);
    free(table);
}

// Demonstration
int main() {
    printf("=== Separate Chaining Hash Table Demo ===\n\n");

    HashTable* table = create_hash_table(TABLE_SIZE);

    // Insert operations
    insert(table, "apple", 100);
    insert(table, "banana", 200);
    insert(table, "orange", 300);
    insert(table, "grape", 400);
    insert(table, "mango", 500);

```

```

    insert(table, "peach", 600);

    display(table);

    // Search operations
    int value;
    if (search(table, "banana", &value)) {
        printf("Found: banana = %d\n", value);
    }

    if (!search(table, "cherry", &value)) {
        printf("Not found: cherry\n");
    }
    printf("\n");

    // Update operation
    insert(table, "apple", 150); // Update existing key

    // Delete operation
    delete(table, "orange");
    display(table);

    // Demonstrate collision
    printf("=== Testing Collision Handling ===\n");
    // These keys may hash to same bucket
    insert(table, "test1", 111);
    insert(table, "test2", 222);
    insert(table, "test3", 333);

    display(table);

    free_hash_table(table);
    return 0;
}

```

Implementation 2: Open Addressing - Linear Probing in C

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

#define TABLE_SIZE 10
#define DELETED_MARKER ((char*)-1) // Tombstone for lazy deletion

typedef struct {
    char* key;
    int value;
} Entry;

typedef struct {
    Entry** entries;

```

```

    int size;
    int count;
} HashTableLP;

// Hash function
unsigned long hash_function_lp(const char* str, int table_size) {
    unsigned long hash = 5381;
    int c;

    while ((c = *str++)) {
        hash = ((hash << 5) + hash) + c;
    }

    return hash % table_size;
}

// Create hash table
HashTableLP* create_hash_table_lp(int size) {
    HashTableLP* table = (HashTableLP*)malloc(sizeof(HashTableLP));
    if (!table) return NULL;

    table->size = size;
    table->count = 0;
    table->entries = (Entry**)calloc(size, sizeof(Entry*));

    return table;
}

// Create entry
Entry* create_entry(const char* key, int value) {
    Entry* entry = (Entry*)malloc(sizeof(Entry));
    if (!entry) return NULL;

    entry->key = strdup(key);
    entry->value = value;
    return entry;
}

// Insert with linear probing
bool insert_lp(HashTableLP* table, const char* key, int value) {
    if (table->count >= table->size * 0.7) {
        printf("Warning: Load factor too high (>0.7)\n");
        return false;
    }

    unsigned long index = hash_function_lp(key, table->size);
    unsigned long original_index = index;
    int probe_count = 0;

    while (table->entries[index] != NULL &&
           table->entries[index] != DELETED_MARKER) {

        // Key already exists - update value
        if (strcmp(table->entries[index]->key, key) == 0) {
            table->entries[index]->value = value;
            printf("Updated: %s = %d at index %lu\n", key, value, index);

```

```

        return true;
    }

    // Linear probing: move to next slot
    index = (index + 1) % table->size;
    probe_count++;

    // Table is full (all slots checked)
    if (index == original_index) {
        printf("Hash table is full!\n");
        return false;
    }
}

// Found empty or deleted slot
Entry* new_entry = create_entry(key, value);
if (!new_entry) return false;

table->entries[index] = new_entry;
table->count++;

printf("Inserted: %s = %d at index %lu (probes: %d)\n",
       key, value, index, probe_count);
return true;
}

// Search with linear probing
bool search_lp(HashTableLP* table, const char* key, int* value) {
    unsigned long index = hash_function_lp(key, table->size);
    unsigned long original_index = index;
    int probe_count = 0;

    while (table->entries[index] != NULL) {
        // Skip deleted markers
        if (table->entries[index] != DELETED_MARKER) {
            if (strcmp(table->entries[index]->key, key) == 0) {
                *value = table->entries[index]->value;
                printf("Found %s after %d probes\n", key, probe_count);
                return true;
            }
        }
        index = (index + 1) % table->size;
        probe_count++;

        if (index == original_index) break;
    }

    return false;
}

// Delete with lazy deletion (tombstone)
bool delete_lp(HashTableLP* table, const char* key) {
    unsigned long index = hash_function_lp(key, table->size);
    unsigned long original_index = index;

```

```

while (table-&entries[index] != NULL) {
    if (table-&entries[index] != DELETED_MARKER) {
        if (strcmp(table-&entries[index]-&key, key) == 0) {
            free(table-&entries[index]-&key);
            free(table-&entries[index]);
            table-&entries[index] = DELETED_MARKER;
            table-&count--;
            printf("Deleted: %s (marked as deleted)\n", key);
            return true;
        }
    }

    index = (index + 1) % table-&size;
    if (index == original_index) break;
}

return false;
}

// Display hash table
void display_lp(HashTableLP* table) {
    printf("\n=== Hash Table (Linear Probing) ===\n");
    printf("Size: %d, Count: %d, Load Factor: %.2f\n\n",
        table-&size, table-&count,
        (float)table-&count / table-&size);

    for (int i = 0; i < table-&size; i++) {
        printf("Index[%d]: ", i);

        if (table-&entries[i] == NULL) {
            printf("Empty\n");
        } else if (table-&entries[i] == DELETED_MARKER) {
            printf("DELETED\n");
        } else {
            printf("(%s: %d)\n",
                table-&entries[i]-&key,
                table-&entries[i]-&value);
        }
    }
    printf("\n");
}

// Free hash table
void free_hash_table_lp(HashTableLP* table) {
    for (int i = 0; i < table-&size; i++) {
        if (table-&entries[i] != NULL &&
            table-&entries[i] != DELETED_MARKER) {
            free(table-&entries[i]-&key);
            free(table-&entries[i]);
        }
    }
    free(table-&entries);
    free(table);
}

// Demonstration

```

```

int main() {
    printf("=== Linear Probing Hash Table Demo ===\n\n");

    HashTableLP* table = create_hash_table_lp(TABLE_SIZE);

    // Insert operations
    insert_lp(table, "apple", 100);
    insert_lp(table, "banana", 200);
    insert_lp(table, "orange", 300);
    insert_lp(table, "grape", 400);

    display_lp(table);

    // Demonstrate collision and probing
    printf("=== Inserting keys that may collide ===\n");
    insert_lp(table, "apricot", 500); // May collide with apple
    insert_lp(table, "blueberry", 600);

    display_lp(table);

    // Search operations
    int value;
    if (search_lp(table, "grape", &value)) {
        printf("Search result: grape = %d\n\n", value);
    }

    // Delete and show tombstone
    delete_lp(table, "banana");
    display_lp(table);

    // Insert after deletion (reuse deleted slot)
    insert_lp(table, "cherry", 700);
    display_lp(table);

    free_hash_table_lp(table);
    return 0;
}

```

Implementation 3: Open Addressing - Quadratic Probing in C

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

#define TABLE_SIZE 11 // Prime number recommended
#define DELETED_MARKER ((char*)-1)

typedef struct {
    char* key;
    int value;
} EntryQP;

```



```

typedef struct {
    EntryQP** entries;
    int size;
    int count;
} HashTableQP;

// Hash function
unsigned long hash_function_qp(const char* str, int table_size) {
    unsigned long hash = 5381;
    int c;

    while ((c = *str++)) {
        hash = ((hash << 5) + hash) + c;
    }

    return hash % table_size;
}

// Create hash table
HashTableQP* create_hash_table_qp(int size) {
    HashTableQP* table = (HashTableQP*)malloc(sizeof(HashTableQP));
    if (!table) return NULL;

    table->size = size;
    table->count = 0;
    table->entries = (EntryQP**)calloc(size, sizeof(EntryQP));

    return table;
}

// Create entry
EntryQP* create_entry_qp(const char* key, int value) {
    EntryQP* entry = (EntryQP*)malloc(sizeof(EntryQP));
    if (!entry) return NULL;

    entry->key = strdup(key);
    entry->value = value;
    return entry;
}

// Insert with quadratic probing
bool insert_qp(HashTableQP* table, const char* key, int value) {
    if (table->count >= table->size * 0.5) {
        printf("Warning: Load factor too high for quadratic probing\n");
        return false;
    }

    unsigned long index = hash_function_qp(key, table->size);
    unsigned long original_index = index;
    int i = 0;

    while (i < table->size) {
        unsigned long probe_index = (original_index + i * i) % table->size;

        if (table->entries[probe_index] == NULL ||

```

```

        table-&entries[probe_index] == DELETED_MARKER) {

    EntryQP* new_entry = create_entry_qp(key, value);
    if (!new_entry) return false;

    table-&entries[probe_index] = new_entry;
    table-&count++;

    printf("Inserted: %s = %d at index %lu (i=%d)\n",
           key, value, probe_index, i);
    return true;
}

// Update existing key
if (strcmp(table-&entries[probe_index]-&key, key) == 0) {
    table-&entries[probe_index]-&value = value;
    printf("Updated: %s = %d at index %lu\n", key, value, probe_index);
    return true;
}

i++;
}

printf("Failed to insert %s: table full or no slot found\n", key);
return false;
}

// Search with quadratic probing
bool search_qp(HashTableQP* table, const char* key, int* value) {
    unsigned long original_index = hash_function_qp(key, table-&size);
    int i = 0;

    while (i < table-&size) {
        unsigned long probe_index = (original_index + i * i) % table-&size;

        if (table-&entries[probe_index] == NULL) {
            return false; // Key not found
        }

        if (table-&entries[probe_index] != DELETED_MARKER) {
            if (strcmp(table-&entries[probe_index]-&key, key) == 0) {
                *value = table-&entries[probe_index]-&value;
                printf("Found %s at index %lu (i=%d)\n", key, probe_index, i);
                return true;
            }
        }

        i++;
    }

    return false;
}

// Delete with lazy deletion
bool delete_qp(HashTableQP* table, const char* key) {
    unsigned long original_index = hash_function_qp(key, table-&size);

```

```

int i = 0;

while (i < table->size) {
    unsigned long probe_index = (original_index + i * i) % table->size;

    if (table->entries[probe_index] == NULL) {
        return false;
    }

    if (table->entries[probe_index] != DELETED_MARKER) {
        if (strcmp(table->entries[probe_index]->key, key) == 0) {
            free(table->entries[probe_index]->key);
            free(table->entries[probe_index]);
            table->entries[probe_index] = DELETED_MARKER;
            table->count--;
            printf("Deleted: %s\n", key);
            return true;
        }
    }

    i++;
}

return false;
}

// Display
void display_qp(HashTableQP* table) {
    printf("\n=== Hash Table (Quadratic Probing) ===\n");
    printf("Size: %d, Count: %d, Load Factor: %.2f\n\n",
        table->size, table->count,
        (float)table->count / table->size);

    for (int i = 0; i < table->size; i++) {
        printf("Index[%d]: ", i);

        if (table->entries[i] == NULL) {
            printf("Empty\n");
        } else if (table->entries[i] == DELETED_MARKER) {
            printf("DELETED\n");
        } else {
            printf("(%s: %d)\n",
                table->entries[i]->key,
                table->entries[i]->value);
        }
    }
    printf("\n");
}

// Free
void free_hash_table_qp(HashTableQP* table) {
    for (int i = 0; i < table->size; i++) {
        if (table->entries[i] != NULL &&
            table->entries[i] != DELETED_MARKER) {
            free(table->entries[i]->key);
            free(table->entries[i]);
        }
    }
}

```

```

    }
}
free(table-&gt;entries);
free(table);
}

// Demonstration
int main() {
    printf("=== Quadratic Probing Hash Table Demo ===\n\n");

    HashTableQP* table = create_hash_table_qp(TABLE_SIZE);

    insert_qp(table, "apple", 100);
    insert_qp(table, "banana", 200);
    insert_qp(table, "orange", 300);
    insert_qp(table, "grape", 400);
    insert_qp(table, "mango", 500);

    display_qp(table);

    int value;
    search_qp(table, "orange", &value);

    delete_qp(table, "banana");
    display_qp(table);

    insert_qp(table, "peach", 600);
    display_qp(table);

    free_hash_table_qp(table);
    return 0;
}

```

Implementation 4: Double Hashing in C

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

#define TABLE_SIZE 11 // Prime number
#define PRIME 7 // Secondary prime (< TABLE_SIZE)
#define DELETED_MARKER ((char*)-1)

typedef struct {
    char* key;
    int value;
} EntryDH;

typedef struct {
    EntryDH** entries;
    int size;
    int count;
} HashTableDH;

```

```

// Primary hash function
unsigned long hash1(const char* str, int table_size) {
    unsigned long hash = 5381;
    int c;

    while ((c = *str++)) {
        hash = ((hash << 5) + hash) + c;
    }

    return hash % table_size;
}

// Secondary hash function (must not return 0)
unsigned long hash2(const char* str, int prime) {
    unsigned long hash = 0;
    int c;

    while ((c = *str++)) {
        hash += c;
    }

    // Ensure result is never 0 and relatively prime to table_size
    return (prime - (hash % prime));
}

// Create hash table
HashTableDH* create_hash_table_dh(int size) {
    HashTableDH* table = (HashTableDH*)malloc(sizeof(HashTableDH));
    if (!table) return NULL;

    table->size = size;
    table->count = 0;
    table->entries = (EntryDH**)calloc(size, sizeof(EntryDH));

    return table;
}

// Create entry
EntryDH* create_entry_dh(const char* key, int value) {
    EntryDH* entry = (EntryDH*)malloc(sizeof(EntryDH));
    if (!entry) return NULL;

    entry->key = strdup(key);
    entry->value = value;
    return entry;
}

// Insert with double hashing
bool insert_dh(HashTableDH* table, const char* key, int value) {
    if (table->count >= table->size * 0.7) {
        printf("Warning: Load factor too high\n");
        return false;
    }

    unsigned long h1 = hash1(key, table->size);

```

```

unsigned long h2 = hash2(key, PRIME);
unsigned long index = h1;
int i = 0;

printf("Inserting %s: h1=%lu, h2=%lu\n", key, h1, h2);

while (i < table->size) {
    index = (h1 + i * h2) % table->size;

    if (table->entries[index] == NULL ||
        table->entries[index] == DELETED_MARKER) {

        EntryDH* new_entry = create_entry_dh(key, value);
        if (!new_entry) return false;

        table->entries[index] = new_entry;
        table->count++;

        printf("  Inserted at index %lu (probe %d)\n", index, i);
        return true;
    }

    if (strcmp(table->entries[index]->key, key) == 0) {
        table->entries[index]->value = value;
        printf("  Updated at index %lu\n", index);
        return true;
    }

    i++;
}

printf("  Failed to insert: table full\n");
return false;
}

// Search with double hashing
bool search_dh(HashTableDH* table, const char* key, int* value) {
    unsigned long h1 = hash1(key, table->size);
    unsigned long h2 = hash2(key, PRIME);
    int i = 0;

    while (i < table->size) {
        unsigned long index = (h1 + i * h2) % table->size;

        if (table->entries[index] == NULL) {
            return false;
        }

        if (table->entries[index] != DELETED_MARKER) {
            if (strcmp(table->entries[index]->key, key) == 0) {
                *value = table->entries[index]->value;
                printf("Found %s at index %lu (probe %d)\n", key, index, i);
                return true;
            }
        }
    }
}

```

```

        i++;
    }

    return false;
}

// Delete
bool delete_dh(HashTableDH* table, const char* key) {
    unsigned long h1 = hash1(key, table->size);
    unsigned long h2 = hash2(key, PRIME);
    int i = 0;

    while (i < table->size) {
        unsigned long index = (h1 + i * h2) % table->size;

        if (table->entries[index] == NULL) {
            return false;
        }

        if (table->entries[index] != DELETED_MARKER) {
            if (strcmp(table->entries[index]->key, key) == 0) {
                free(table->entries[index]->key);
                free(table->entries[index]);
                table->entries[index] = DELETED_MARKER;
                table->count--;
                printf("Deleted: %s\n", key);
                return true;
            }
        }

        i++;
    }

    return false;
}

// Display
void display_dh(HashTableDH* table) {
    printf("\n=== Hash Table (Double Hashing) ===\n");
    printf("Size: %d, Count: %d, Load Factor: %.2f\n\n",
        table->size, table->count,
        (float)table->count / table->size);

    for (int i = 0; i < table->size; i++) {
        printf("Index[%d]: ", i);

        if (table->entries[i] == NULL) {
            printf("Empty\n");
        } else if (table->entries[i] == DELETED_MARKER) {
            printf("DELETED\n");
        } else {
            printf("(%s: %d)\n",
                table->entries[i]->key,
                table->entries[i]->value);
        }
    }
}

```

```

        printf("\n");
    }

    // Free
    void free_hash_table_dh(HashTableDH* table) {
        for (int i = 0; i < table->size; i++) {
            if (table->entries[i] != NULL &&
                table->entries[i] != DELETED_MARKER) {
                free(table->entries[i]->key);
                free(table->entries[i]);
            }
        }
        free(table->entries);
        free(table);
    }

    // Demonstration
    int main() {
        printf("=== Double Hashing Table Demo ===\n\n");

        HashTableDH* table = create_hash_table_dh(TABLE_SIZE);

        insert_dh(table, "apple", 100);
        insert_dh(table, "banana", 200);
        insert_dh(table, "orange", 300);
        insert_dh(table, "grape", 400);
        insert_dh(table, "mango", 500);
        insert_dh(table, "peach", 600);

        display_dh(table);

        int value;
        search_dh(table, "grape", &value);

        delete_dh(table, "banana");
        display_dh(table);

        free_hash_table_dh(table);
        return 0;
    }

```

Summary and Best Practices

Choosing the Right Technique

Use Separate Chaining when:

- Load factor > 0.7 expected
- Frequent deletions required
- Memory for pointers acceptable
- Simple implementation preferred

- Variable data size

Use Linear Probing when:

- Cache performance critical
- Load factor < 0.5
- Simple implementation
- Memory constrained (no pointers)

Use Quadratic Probing when:

- Reducing primary clustering important
- Table size is prime
- Load factor < 0.5

Use Double Hashing when:

- Best performance needed
- Minimal clustering desired
- Can afford two hash computations

Performance Tips

1. **Choose Prime Table Sizes:** Reduces clustering, especially for open addressing
2. **Monitor Load Factor:** Resize when $\alpha > 0.7-0.8$
3. **Quality Hash Functions:** Use proven algorithms (djb2, MurmurHash, etc.)
4. **Consider Cache:** Open addressing better for cache-sensitive applications
5. **Lazy Deletion:** Use tombstones in open addressing
6. **Rehashing:** Implement dynamic resizing for production systems

This comprehensive guide provides the theoretical foundation and practical implementations necessary for mastering hash tables and their collision resolution techniques in data structures and algorithms.

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16]

✱

1. <https://www.geeksforgeeks.org/dsa/hash-table-data-structure/>
2. <https://www.geeksforgeeks.org/dsa/collision-resolution-techniques/>
3. <https://www.w3resource.com/c-programming-exercises/hash/index.php>
4. <https://www.programiz.com/dsa/hash-table>
5. https://en.wikipedia.org/wiki/Open_addressing
6. <https://www.geeksforgeeks.org/dsa/implementation-of-hash-table-in-c-using-separate-chaining/>
7. <https://favtutor.com/blogs/hashing-in-data-structure>
8. <https://www.geeksforgeeks.org/dsa/open-addressing-collision-handling-technique-in-hashing/>

9. <https://www.digitalocean.com/community/tutorials/hash-table-in-c-plus-plus>
10. <https://www.interviewcake.com/concept/java/hash-map>
11. <https://visualgo.net/en/hashtable>
12. <https://www.geeksforgeeks.org/dsa/collision-resolution-techniques/>
13. <https://www.youtube.com/watch?v=FsfRsGFHuv4>
14. https://www.w3schools.com/dsa/dsa_theory_hashtables.php
15. <https://cexpertvision.com/collision-resolution-techniques/>
16. <https://www.log2base2.com/algorithms/searching/hashing-in-c-data-structure.html>